

OPERATOR_GUIDE

HelixLLM Operator Guide — Local LLM Stack on the RTX 5090

Status	ACTIVE · Revision 1 · Track (T1/main)
Date	2026-07-06
Branch	feature/helixllm-full-extension
Scope	Running the local HelixLLM inference stack (llama.cpp CUDA router image + Qwen3-Coder-30B fleet) on a single RTX 5090, and pointing CLI agents / HelixAgent at it.
Grounding (§11.4.6)	Every capability claim below cites captured evidence from this session. Nothing here is aspirational. Where a feature is design-only or planned, it is labelled as such.

Anti-bluff boundary (§11.4 / §11.4.6). This guide documents ONLY what was proven at runtime in this session with captured evidence. The VRAM residency broker is **design-only** (§9); vision / image / video are **planned, not shipped** (§10). Both are called out explicitly so an operator is never misled.

1. Prerequisites

The stack was proven on this exact host (docs/research/07.2026/00_master/RESUME.md live-state anchors):

Requirement	Proven value	Notes
GPU	NVIDIA RTX 5090, 32 GB (Blackwell, sm_120)	The router image is compiled for native sm_120 SASS — see §2.
NVIDIA driver	570.169 (CUDA 12.8)	

Requirement	Proven value	Notes
		CUDA 12.8 is the first CUDA branch with Blackwell sm_120 support; 12.6.x does not build native sm_120 kernels (container/Containerfile.llamacpp-router:5-6).
Container runtime	podman 5.7.1, rootless	Rootful Docker / sudo for containers is forbidden (§11.4.161).
nvidia-container-toolkit + CDI	Installed; CDI spec at ~/.config/cdi/nvidia.yaml	Rootless podman wired via ~/.config/containers/containers.conf → cdi_spec_dirs = ["~/.config/cdi"]. GPU passthrough proven (docs/qa/phase0_gpu/).
Host CPU / RAM	64 cores / 251 GiB (ALT Workstation 11.1)	The run line pins 14 inference threads; adjust for your host.
Models directory	~/models/ on the host	Mounted read-only into the container at /models (see §3).

GPU passthrough smoke test (proves the toolkit + CDI wiring before you run any model):

```
podman run --rm --device nvidia.com/gpu=all --security-
    opt=label=disable \
    localhost/helixllm/llamacpp-router:cuda12.8-sm120 nvidia-smi
```

This must print the RTX 5090. If it does not, the CDI wiring is the blocker, not the model.

2. Build the router image

The router image is the single build artefact this stack runs on. It was **built and proven** in this session (image localhost/helixllm/llamacpp-router:cuda12.8-sm120; Containerfile container/Containerfile.llamacpp-router).

What the image contains (all verified in the Containerfile, cited inline there):

- **Latest llama.cpp** from upstream `ggml-org/llama.cpp` (bleeding edge, per operator mandate) built as static, self-contained binaries.
- **Native Blackwell sm_120** SASS — `-DCMAKE_CUDA_ARCHITECTURES=120` (no PTX JIT fallback), required for the RTX 5090.
- **OpenSSL + curl** (`-DLLAMA_CURL=ON -DLLAMA_OPENSSL=ON, libcurl4-openssl-dev + libssl-dev`) — enables HTTPS GGUF download via `-hf` (§5). Without OpenSSL, `-hf` fails at runtime with *“HTTPS is not supported”* (verified 2026-07-06).
- **RPC server** (`ggml-rpc-server`, gated on `-DGGML_RPC=ON`) shipped at `/usr/local/bin/rpc-server` for future multi-host distributed inference. The copy step **hard-fails** the build if the binary is missing, so RPC can never be silently dropped (§11.4.108 / §11.4.122).
- Flash-Attention kernels for all quant types, F16 CUDA path, and LTO.

Build it:

```
cd submodules/helix_llm
podman build -f container/Containerfile.llamacpp-router \
-t localhost/helixllm/llamacpp-router:cuda12.8-sm120 .
```

Note (build-time libcuda stub). The `-devel` base image ships only a stub `libcuda.so`; the real `libcuda.so.1` is injected at **runtime** by the GPU driver via CDI. The Containerfile symlinks the stub to its SONAME and puts the stubs dir on the linker path so the static link resolves — do not remove those lines (Containerfile.llamacpp-router:20-25,57).

Build-layer evidence: `docs/qa/phase0_build/p0t3_llamacpp_buildproof.log` (via RESUME.md). **Real GPU inference proven** on this image: “2+2” → “4”, 495 tok/s, 1008 MiB VRAM (RESUME.md live-state).

3. Run the coder fleet

This is the **live, operator-testable** service. Qwen3-Coder-30B-A3B serves an OpenAI-compatible API at `http://localhost:18434/v1`.

Model file required on the host: `~/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf` (download via `-hf` — see §5 — or place it manually).

```
podman run -d --name helixllm-coder \
--device nvidia.com/gpu=all \
--security-opt=label=disable \
```

```

--network=host \
-v "$HOME/models:/models:ro" \
localhost/helixllm/llamacpp-router:cuda12.8-sm120 \
-m /models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf \
-ngl 99 \
-c 24576 \
--parallel 8 \
--cont-batching \
-fa on \
--cache-type-k q8_0 \
--cache-type-v q8_0 \
--host 0.0.0.0 \
--port 18434 \
--jinja

```

Flag-by-flag:

Flag	Meaning
-d --name helixllm-coder	Detached; the container name used by podman start/stop/ps.
--device nvidia.com/gpu=all	CDI GPU passthrough (rootless). The whole card is offered.
--security-opt=label=disable	Required for rootless CDI GPU access on this host (SELinux label relabel off for the device nodes).
--network=host	Publishes port 18434 directly on the host so localhost:18434 is reachable. (Without host networking, the port stays inside the container's netns.)
-v "\$HOME/models:/models:ro"	Mounts the host models dir read-only at /models.
-m /models/...Q4_K_M.gguf	The GGUF to load.
-ngl 99	Offload all layers to the GPU.
-c 24576	24k context window.
--parallel 8	8 concurrent inference slots (see §6).
--cont-batching	Continuous batching across the 8 slots.
-fa on	Flash Attention on . GOTCHA: in the latest llama.cpp -fa takes a <i>value</i> — -fa on, not a bare -fa.

Flag	Meaning
--cache-type-k q8_0 / --cache-type-v q8_0	q8_0-quantized KV cache (smaller VRAM footprint).
--host 0.0.0.0 --port 18434	Bind address / port.
--jinja	Use the model's Jinja chat template for OpenAI-style chat formatting.

Measured resident VRAM: ~19.4 GB (19432 MiB, docs/qa/phase2_e2e_20260706/ 01_nvidia_smi_pre.txt / 03_nvidia_smi_during.txt).

Lifecycle:

```
podman ps                # confirm helixllm-coder is Up
podman start helixllm-coder # restart an existing (stopped)
                           container
podman stop helixllm-coder # stop it
podman logs -f helixllm-coder # tail server logs
```

4. OpenAI-compatible endpoints + a working curl

The coder fleet exposes the standard llama-server OpenAI-compatible surface at `http://localhost:18434`. The health check and chat endpoint are the two you need day to day.

Health:

```
curl -s http://localhost:18434/health
```

Real chat completion (this is the proven end-to-end path — a genuine coding task returning genuine code, docs/qa/phase1_fleet/coder_live_e2e_20260706.log):

```
curl -sS http://localhost:18434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf",
    "messages": [
      {"role": "user", "content": "Write a Python function
        is_palindrome(s) that ignores case and non-
        alphanumerics."}
    ]
  }'
```

Proven response for exactly this class of request
(coder_live_e2e_20260706.log): a real `is_palindrome` implementation,
finish: stop, 59 completion tokens in 0.27 s (~220 tok/s **single-stream**). No
stub, no simulation marker.

4.1 The base-URL /v1 gotcha (load-bearing — from the Phase-2 proof)

llama-server's chat endpoint is `POST /v1/chat/completions`. How you configure a client depends on whether the client appends the `/v1/...` suffix itself:

- **Raw curl (you write the full path):** use `http://localhost:18434/v1/chat/completions` — the full path, as shown above.
- **An OpenAI client / library that appends `/v1/chat/completions` to a base URL:** set the base URL to **`http://localhost:18434`** — the base **without** `/v1`.

If you give such a client a base URL of `http://localhost:18434/v1`, it becomes `http://localhost:18434/v1 + /v1/chat/completions = /v1/v1/chat/completions` → **HTTP 404**. This was confirmed both ways (wrong → 404, correct → 200) in `docs/qa/phase2_e2e_20260706/12_endpoint_finding.txt`. This is the single most common misconfiguration when wiring an agent to this stack.

5. Downloading models with -hf (HTTPS)

The router image can download GGUF models directly from Hugging Face over HTTPS using llama.cpp's built-in `-hf` downloader. This is **proven working** — a 469 MB GGUF was downloaded over HTTPS in this session (`docs/qa/phase1_fleet/hf_https_proof_20260706.log`; §11.4.108 verdict at the tail of that log). It works because the image is built with `-DLLAMA_CURL=ON` `-DLLAMA_OPENSSL=ON` and ships `libssl3/libcurl4` (§2); the earlier non-`--hf` P0 image could **not** do this ("HTTPS is not supported").

Example (downloads on first use into the container's HF cache, then serves):

```
podman run -d --name hx-dl \  
  --device nvidia.com/gpu=all --security-opt=label=disable -- \  
  network=host \  
  -v "$HOME/models:/models:rw" \  
  localhost/helixllm/llamacpp-router:cuda12.8-sm120 \  
  -hf Qwen/Qwen2.5-0.5B-Instruct-GGUF:Q4_K_M \  
  -ngl 99 --host 0.0.0.0 --port 18434
```

Cache note (from the proof log). Point each `-hf` invocation at a single-owner cache directory. The proof log captured a self-inflicted rename-error when two invocations shared one HF cache concurrently — the blob still downloaded over HTTPS, but concurrent writers to one cache race. Give distinct download jobs distinct caches (or serialise them). For the primary fleet model, downloading it once to `~/models/` and then serving with `-m` (as in §3) avoids the issue entirely.

6. Concurrency (parallel slots)

The fleet is configured with `--parallel 8 --cont-batching` (§3) — 8 concurrent inference slots with continuous batching.

Proven throughput:

- **Single stream:** ~220 tok/s (59 tokens / 0.27 s, `coder_live_e2e_20260706.log`).
- **8 concurrent agents: 85–96 tok/s each, simultaneously** (RESUME.md live-state — 3× the ≥30 tok/s design target). Real coding output on every slot.

So a team of up to 8 CLI agents can share this one server and each still gets interactive-grade throughput. To change the slot count, adjust `--parallel N` (more slots = more concurrent KV cache = more VRAM; the `q8_0` KV cache keeps the footprint at ~19.4 GB for 8 slots at 24k ctx).

7. Pointing a CLI agent / HelixAgent at the stack

Any OpenAI-compatible agent works. Configure it with:

- **Base URL:** `http://localhost:18434` (base **without** `/v1` — see §4.1).
- **API key:** any non-empty string; the coder fleet (raw llama-server) does not enforce auth on 18434.
- **Model:** `/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf` (or whatever the server reports at `GET /v1/models`).

HelixAgent (proven end-to-end)

HelixAgent's `helixllm.Provider` was driven against this live server in the Phase-2 proof (`docs/qa/phase2_e2e_20260706/RESULTS.md`), returning genuine code (a real `func Add`). The provider is pinned via an environment variable:

```
export HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434
# base, NO /v1
```

The provider hardcodes the /v1/chat/completions suffix, so the pin **must** be the base without /v1 (the §4.1 gotcha — pinning .../v1 yields /v1/v1/... → 404). With the base pin set, HelixAgent issued a real generate and received non-empty code, tokens_used > 0, no bluff markers (evidence 11_green_proof.txt; the RED_MODE=1 baseline with no pin fails with connection refused against the unrunning TLS default — 10_red_baseline.txt).

Reproduce (from RESULTS.md):

```
cd submodules/helix_agent
HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434
RED_MODE=0 \
go test -tags=helixllm_e2e -run
    TestE2E_HelixAgent_To_LiveHelixLLM -v \
./internal/llm/providers/helixllm/
```

HelixAgent over the LAN / VPN (env-var parameterized, host IP + auth)

The helixllm.Provider works as an OpenAI-compatible provider alias **not only on localhost but anywhere on the LAN or VPN**, parameterized entirely by environment variables — no code change, no hardcoded host (CONST-045). The coder fleet already binds 0.0.0.0:18434 (see §4 / §6), so it is reachable at the host's LAN IP the moment the port is open.

Client-target env vars (read by helixllm.Provider.resolveEndpoint):

Var	Meaning	Default
HELIX_LLM_HOST	host/IP of the OpenAI-compatible router to connect to	localhost
HELIX_LLM_PORT	port of that router	18434
HELIX_LLM_API_KEY	optional Bearer key sent as Authorization: Bearer <key>	(none)
HELIX_LLM_LOCAL_OPENAI_ENDPOINT	explicit base URL seam (wins over HOST/PORT)	(none)

Var	Meaning	Default
HELIX_LLM_ENDPOINT	general base URL override	(none)

Endpoint precedence (first non-empty wins): explicit `cfg.Endpoint` → `HELIX_LLM_LOCAL_OPENAI_ENDPOINT` → `HELIX_LLM_ENDPOINT` → `HELIX_LLM_HOST/HELIX_LLM_PORT` composition (`http://${HELIX_LLM_HOST}:${HELIX_LLM_PORT}`) → the TLS :8443 gateway default. Setting only `HELIX_LLM_HOST` inherits port 18434; a server-bind `0.0.0.0` is mapped to localhost for the client target.

Point HelixAgent at the host over the LAN — pick ONE:

```
# (a) HOST/PORT composition – the LAN case, host defaults port
    18434
export HELIX_LLM_HOST=10.6.100.221      # the host's LAN/VPN IP
    (NOT localhost)
export HELIX_LLM_PORT=18434

# (b) or the explicit base-URL seam (equivalent; wins over HOST/
    PORT)
export HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://
    10.6.100.221:18434    # base, NO /v1
```

The /v1 gotcha (load-bearing — see §4.1): the endpoint **MUST** be the BASE `http://<HOST>:18434` **without** `/v1`. The provider appends `/v1/` `chat/completions` itself; a trailing `/v1` would double to `/v1/v1` → 404. As a guard, `normalizeBase` now strips a trailing `/v1` so both `.../18434` and `.../18434/v1` resolve to the base — but write the base to be safe.

API-key auth. When `HELIX_LLM_API_KEY` is set the provider sends `Authorization: Bearer <key>` on every request. The raw coder fleet on 18434 does **not** enforce auth (any/no key is accepted); to require auth, front the model with a key-checking OpenAI server — e.g. `llama-server --host 0.0.0.0 --port 18439 --api-key <key>` (`llama.cpp --api-key` does a byte-for-byte Bearer compare → **401** on a missing/wrong key, **200** with the correct key), or the full HelixLLM gateway on :8443. Proven matrix (evidence docs/qa/helixagent_network_provider_20260707/): no key → 401, wrong key → 401, correct key → 200 + real completion.

Where to put the exports. Export the vars however your host manages secrets: a gitignored `api_keys.sh` sourced from your shell rc (host-local, never committed — see `api_keys.sh.example`), OR the project `.env`. The API key is a secret: never commit it (CONST-042 / §11.4.10). `.env` / `api_keys.sh` are gitignored.

Firewall / reachability. LAN/VPN use requires the coder port reachable from the client host (open 18434 on the server's firewall, or route it over the VPN). Verify with a raw curl before wiring the agent:

```
curl -sS -X POST http://10.6.100.221:18434/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "default", "messages":
    [{"role": "user", "content": "Reply: LANPROOF-
    OK"}], "max_tokens": 16}'
```

Live LAN proof (reproduce). The provider was driven over the LAN interface (10.6.100.221, not localhost) via the HOST/PORT composition, returning genuine code (func Add):

```
cd submodules/helix_agent
HELIX_LLM_HOST=10.6.100.221 HELIX_LLM_PORT=18434 \
go test -tags=helixllm_e2e -run
    TestE2E_HelixAgent_To_LiveHelixLLM_ViaHostPort -v \
./internal/llm/providers/helixllm/
```

The full HelixLLM Go gateway (:8443) — separate, documented contract

There is also a full HelixLLM Go server (cmd/helixllm/main.go) that fronts an **embedded** llama.cpp brain behind a TLS-mandatory, API-key-authenticated OpenAI + Anthropic gateway on https://0.0.0.0:8443. Its complete route inventory, auth wiring, and request/response shapes are documented in docs/API_CONTRACT.md (source-verified from code). Key facts from that contract:

- Base URL https://0.0.0.0:8443; **TLS 1.3 minimum, mandatory** (serving fails immediately if cert/key are empty); serves HTTP/3 (QUIC) + HTTP/2 on the same port.
- OpenAI endpoints under /v1/* (/v1/chat/completions, /v1/completions, /v1/models, /v1/embeddings, ...) require an **API key**; the Anthropic /v1/messages endpoint is also there.
- The embedded-llama toggle is LlamaServerEmbed (HELIX_LLAMA_SERVER_EMBEDDED, default true) — **not** UseLlamaCpp (which does not exist in the code).
- **Security finding (from the contract):** /v1/agents/*, /v1/cache/stats, all /internal/*, /metrics, and /ws are **unauthenticated** as currently wired — only the 7 gateway /v1 LLM endpoints enforce the API key. Do not expose :8443 on an untrusted network until this is addressed.

This guide documents the :18434 coder fleet as the proven operator path; the :8443 gateway is presented from its verified code contract (API_CONTRACT.md), not from a captured live-serving run in this session.

8. Troubleshooting (the real gotchas)

Symptom	Cause	Fix
-fa rejected / server won't start on the FA flag	Latest llama.cpp changed -fa to take a value.	Use -fa on (not a bare -fa).
Client gets HTTP 404 on every chat call	Base URL includes /v1, and the client also appends /v1/chat/completions → /v1/v1/....	Set the client base URL to http://localhost:18434 (no /v1). Raw curl uses the full .../v1/chat/completions. (§4.1)
HelixAgent → connection refused	No endpoint pin set → provider falls back to the (non-running) TLS :8443 default.	export HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434 (base, no /v1).
-hf fails with “ <i>HTTPS is not supported</i> ”	Using an image built without OpenSSL/curl (the old P0 image).	Use localhost/helixllm/llamacpp-router:cuda12.8-sm120, built with -DLLAMA_CURL=ON -DLLAMA_OPENSSL=ON.
-hf rename / downloadInProgress error	Two -hf jobs sharing one HF cache concurrently.	Give each job its own cache dir, or download once to ~/models/ and serve with -m. (§5)
nvidia-smi in container fails / no GPU	CDI wiring, not the model.	Run the §1 smoke test; check nvidia-container-toolkit, ~/.config/cdi/nvidia.yaml, and cdi_spec_dirs in containers.conf.
		Include --network=host in the run line (§3).

Symptom	Cause	Fix
localhost:18434 unreachable from host	Container not on host network.	
Model runs on CPU / slow	Layers not offloaded, or a non-sm_120 image.	Ensure -ngl 99 and that you built with -DCMAKE_CUDA_ARCHITECTURES=120 (§2).

9. VRAM residency broker — DESIGN ONLY (not shipped)

The programme plans a **VRAM residency broker** to co-host ~10 GPU services on the single 32 GB card (keep the coder fleet resident, swap mid-tier models, run heavy generators single-owner, never OOM). This is a **design spike only — no broker code exists yet**. The design (residency tiers, admission API, eviction policy, anti-bluff acceptance criteria) is in [docs/VRAM_BROKER.md](#). Do not expect broker behaviour from the current stack: today the coder fleet is a single resident model and cross-service VRAM arbitration is manual.

10. Planned capabilities — NOT shipped

Per the programme plan, the following are **planned, not yet available** in this stack (documented here so no operator assumes they work today):

- **Vision / image understanding (VLM), image generation, video generation** — planned warm-swappable / burst tiers in the VRAM broker design (VRAM_BROKER.md §2), not implemented.
- **Multi-host distributed inference** — the RPC server binary ships in the image (/usr/local/bin/rpc-server, §2) but multi-host serving is not yet wired or proven at runtime.
- **Cognee / vector memory write path** — an honest §11.4.3 SKIP in Phase-2 (real upstream Cognee bug + unwired repo path; RESULTS.md §3).

Sources verified

Every claim in this guide is grounded in captured evidence from this session (2026-07-06), not external memory. Primary sources (paths relative to `submodules/helix_llm/` unless noted):

- `docs/qa/phase1_fleet/coder_live_e2e_20260706.log` — live coder end-to-end (is_palindrome, 59 tok / 0.27 s, finish=stop).
- `docs/qa/phase1_fleet/hf_https_proof_20260706.log` — -hf HTTPS download proof (469 MB GGUF) + \$11.4.108 verdict.
- `.../docs/qa/phase2_e2e_20260706/RESULTS.md` (main repo `docs/qa/`) — HelixAgent → live HelixLLM real generate; Postgres/Redis persistence; the base-URL /v1 gotcha (`12_endpoint_finding.txt`); VRAM 19432 MiB (`01/03_nvidia_smi*`).
- `container/Containerfile.llamacpp-router` — the built router image (sm_120, OpenSSL -hf, RPC).
- `docs/API_CONTRACT.md` — the :8443 Go gateway contract (routes, TLS, auth, security finding), source-verified from code.
- `docs/VRAM_BROKER.md` — VRAM broker design spike (design-only).
- `.../docs/research/07.2026/00_master/RESUME.md` (main repo) — live-state anchors: host spec, image id, run line, 8-concurrent throughput, GPU stack.

LAN/VPN + auth section (added 2026-07-07) — evidence in main repo `docs/qa/helixagent_network_provider_20260707/:30_live_lan_hostport.txt` (provider driven via `HELIX_LLM_HOST=10.6.100.221` → real func Add over the LAN), `20_auth_matrix_ephemeral.txt` (no-key → 401, wrong-key → 401, correct-key → 200), `21_provider_keyed_lan.txt` (provider + `HELIX_LLM_API_KEY` → keyed server over LAN → 200). `llama.cpp --api-key/-` host behaviour cross-referenced against the official server docs: - <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md> (verified 2026-07-07) — `--api-key` (Bearer auth, 401 on missing/wrong), `--host` default `127.0.0.1, 0.0.0.0` binds all interfaces, `--port` default 8080. - <https://markaicode.com/errors/llamacpp-api-key-invalid-fix-production/> (verified 2026-07-07) — Authorization: Bearer <key> byte-for-byte compare → 401 on any mismatch.

Honest boundary (§11.4.6 / §11.4.99): external-tool syntax (`llama.cpp -fa on / -hf`, podman CDI flags) is documented exactly as it behaved in this session's captured logs, not re-fetched from upstream docs. Throughput figures are the measured values in the cited logs; the single-stream ~220 tok/s comes from the Phase-1 e2e log (a `--jinja/q8_0-KV` run recorded ~322 tok/s in `RESUME.md` — both are cited rather than averaged).